

Friedrich-Alexander-Universität
Erlangen-Nürnberg (FAU)

NHR@FAU

Assigmnent 10: Stencils, slow computing

Programming Techniques for Supercomputers (PTfS)

Muhammad Omar
Student IdM: to37gulo

Gehan Younes
Student IdM: qi39dyry

Ahmed Mohamed
Student IdM: gi10dafy

July 15, 2026

1 Stencils (40 Credits)

Consider the stencil update

```
#pragma omp parallel for schedule(static)
for(int j=1; j<N-2; ++j)
for(int i=1; i<M-1; ++i)
y[j][i] = c * (x[j][i-1]
+ x[j+1][i]
+ x[j+2][i]
+ x[j+2][i-1]);
```

(a) Layer condition

For a fixed outer-loop index (j), the stencil accesses three rows of the input array:

$$x[j][*], \qquad x[j+1][*], \qquad x[j+2][*].$$

When (j) is incremented, two of these rows are reused. Therefore, in order to exploit this temporal reuse, three complete rows of (x) must fit into the considered cache level.

Since each element is stored in double precision, one row requires

$$8M \text{ bytes.}$$

Hence, the relevant layer condition is

$$3M \cdot 8 < \frac{C_{\text{eff}}}{2},$$

where (C_{eff}) is the effective cache capacity available to the thread. The factor (1/2) represents the usual safety margin used in layer-condition analysis.

For a cache shared among (T) threads, the condition can equivalently be written as

$$3M \cdot 8 \cdot T < \frac{C}{2}.$$

Therefore,

$$\boxed{3M \cdot 8 < \frac{C_{\text{eff}}}{2}}.$$

(b) Best-case and worst-case in-memory code balance

In the best case, the layer condition is fulfilled and the required rows remain in cache. Thus, each element of the input array (x) only needs to be loaded once from memory. This results in

$$8 \text{ bytes/LUP}$$

of memory traffic for (x).

For the output array (y), ordinary write-allocate stores generate

$$8 \text{ bytes/LUP}$$

for the write allocation and another

$$8 \text{ bytes/LUP}$$

for the eventual write-back to memory. Therefore, the total best-case code balance is

$$B_{\text{best}} = 8 + 8 + 8 = 24 \text{ bytes/LUP}.$$

In the worst case, the layer condition is not fulfilled, so the three input rows must be loaded independently from memory. The two accesses

$$x[j+2][i] \qquad \text{and} \qquad x[j+2][i-1]$$

belong to the same row and benefit from spatial locality. Therefore, they form only one memory stream.

Consequently, the input traffic is

$$3 \cdot 8 = 24 \text{ bytes/LUP}.$$

Adding the write allocation and write-back traffic for (y), the total worst-case code balance becomes

$$B_{\text{worst}} = 24 + 16 = 40 \text{ bytes/LUP}.$$

Overall,

$$\boxed{B_{\text{best}} = 24 \text{ bytes/LUP}, \qquad B_{\text{worst}} = 40 \text{ bytes/LUP}}.$$

(c) Absolute upper performance limit on one Fritz socket

The maximum attainable performance for a memory-bound stencil is given by

$$P_{\max} = \frac{b_{\text{mem}}}{B_c},$$

where (b_{mem}) is the sustainable memory bandwidth and (B_c) is the code balance.

For one Fritz socket, the memory bandwidth is

$$b_{\text{mem}} = 160 \text{ GB/s}.$$

Using the best-case code balance,

$$B_c = 24 \text{ bytes/LUP},$$

the absolute upper performance limit is

$$P_{\max} = \frac{160 \times 10^9}{24} = 6.67 \times 10^9 \text{ LUP/s}.$$

Therefore,

$$\boxed{P_{\max} \approx 6.67 \text{ GLUP/s}}.$$

If the layer condition is not fulfilled, the corresponding upper limit would be

$$P_{\max,\text{worst}} = \frac{160}{40} 4.0 \text{ GLUP/s}.$$

The Fritz nodes are based on Intel Ice Lake processors. Each core has approximately

$$1.25 \text{ MB}$$

of private L2 cache, while the socket provides

$$54 \text{ MB}$$

of shared L3 cache across (36) cores. Thus, the proportional L3 cache capacity per core is

$$\frac{54}{36} = 1.5 \text{ MB/core}.$$

Since the L3 cache acts as an exclusive victim cache, the effective cache capacity per core can be approximated as

$$C_{\text{eff}} = 1.25 + 1.5 = 2.75 \text{ MB}.$$

Therefore, the layer condition becomes

$$3M \cdot 8 < \frac{2.75 \text{ MB}}{2}.$$

Using binary units,

$$M < \frac{1.375 \cdot 2^{20}}{24} \approx 6.0 \times 10^4.$$

Hence, approximately,

$$\boxed{M \lesssim 60000}$$

is required for the three-row layer condition to hold per core.

(d) Layer condition for the modified stencil

Now consider the modified stencil

```
#pragma omp parallel for schedule(static)
for(int j=1; j<N-2; ++j)
    for(int i=1; i<M-1; ++i)
        y[j][i] = c * (x[j][i-1]
                     + x[j+2][i]
                     + x[j+2][i-1]);
```

For a fixed value of j, the stencil accesses the two rows

$$x[j][*] \qquad \text{and} \qquad x[j+2][*].$$

However, the layer condition is determined by the complete vertical extent of the stencil. The row offsets are

$$\{0, +2\}.$$

Hence, the vertical extent is

$$2 - 0 + 1 = 3$$

rows.

This can also be seen directly from the lifetime of a row. Consider row $x[r][*]$. It is first loaded at outer-loop iteration

$$j = r - 2,$$

where it appears as $x[j+2][*]$, and it is used for the last time at

$$j = r,$$

where it appears as $x[j][*]$.
Therefore, row r must remain resident during the iterations

$$[r-2, r].$$

At outer-loop iteration j, the rows that may still be required are thus

$$x[j][*], \qquad x[j+1][*], \qquad x[j+2][*].$$

Hence, only three complete rows must fit into the cache. The layer condition is

$$\boxed{3M \cdot 8 < \frac{C_{\text{eff}}}{2}}$$

or, for a cache shared by T threads,

$$\boxed{3M \cdot 8 \cdot T < \frac{C}{2}.}$$

Thus, although the modified stencil accesses fewer points per lattice update, its layer condition is unchanged from part (a): three rows must remain in cache.

2 Slow Computing (35 Credits)

The serial execution time of the program is

$$T(1) = T_s + T_p,$$

where (T_s) is the runtime of the inherently serial part and (T_p) is the runtime of the perfectly parallelizable part. For (N) workers, an additional communication or synchronization overhead ($T_c(N)$) is introduced.

(a) Strong-scaling speedup model

We define the dimensionless quantities

$$s = \frac{T_s}{T(1)}, \qquad p = \frac{T_p}{T(1)}, \qquad c(N) = \frac{T_c(N)}{T(1)}.$$

Since

$$T(1) = T_s + T_p,$$

we have

$$s + p = 1.$$

For (N) workers, the execution time is

$$T(N) = T_s + \frac{T_p}{N} + T_c(N).$$

Dividing by (T(1)) gives

$$\frac{T(N)}{T(1)} = s + \frac{p}{N} + c(N).$$

Therefore, the strong-scaling speedup is

$$S(N) = \frac{T(1)}{T(N)}$$

$$\boxed{\frac{1}{s + \frac{p}{N} + c(N)}}.$$

Without communication overhead, i.e., (c(N)=0), this reduces to the usual form of Amdahl's Law.

(b) Effect of slowing down the computation

Assume that all parts related to code execution are slowed down by a factor

$$0 < \sigma < 1.$$

For example, ($\sigma = 0.5$) means that the computational performance is reduced to half of its original value. Therefore, the execution times of the serial and parallel computational parts increase by a factor ($1/\sigma$):

$$T_s \to \frac{T_s}{\sigma}, \qquad T_p \to \frac{T_p}{\sigma}.$$

However, the communication time ($\mathrm{T}_c(N)$)$remainsunchanged.$
The single-worker runtime of the slowed program is therefore

$$T(1, \sigma) = \frac{T_s + T_p}{\sigma} = \frac{T(1)}{\sigma}.$$

For (N) workers, the runtime becomes

$$T(N, \sigma) = \frac{T_s}{\sigma} + \frac{T_p}{\sigma N} + T_c(N).$$

Using the dimensionless quantities defined above,

$$T(N, \sigma) = T(1) \left[\frac{s}{\sigma} + \frac{p}{\sigma N} + c(N) \right].$$

The speedup is measured relative to the slowed single-worker execution:

$$S(N, \sigma) = \frac{T(1, \sigma)}{T(N, \sigma)}.$$

Hence,

$$S(N, \sigma) = \frac{1/\sigma}{\frac{s}{\sigma} + \frac{p}{\sigma N} + c(N)}.$$

Multiplying numerator and denominator by (σ) gives

$$\boxed{S(N, \sigma) = \frac{1}{s + \frac{p}{N} + \sigma c(N)}}.$$

This result shows that decreasing (σ), i.e., making the computational part of the program slower, reduces the relative importance of the communication overhead:

$$c(N) \to \sigma c(N).$$

Therefore, for

$$c(N) > 0,$$

a smaller value of (σ) leads to a larger measured speedup. In this sense, the humorous statement that compiling with `-O0` can improve the scaling of a program is indeed justified.

The only prerequisite is that there must be a nonzero overhead whose absolute runtime is not slowed down together with the computation. In this model, this is the communication or synchronization overhead ($T_c(N)$). If

$$c(N) = 0,$$

then

$$S(N, \sigma) = \frac{1}{s + \frac{p}{N}},$$

which is independent of (σ). Similarly, if communication were slowed down by exactly the same factor as the computation, the speedup would also remain unchanged.

Thus, better speedup does not necessarily imply better absolute performance. A deliberately slowed-down program may exhibit a larger speedup simply because its single-worker reference execution has become slower while the communication overhead remains constant.

Consequently, speedup alone is not a sufficient metric for assessing the quality of a parallel implementation. A program can have excellent apparent scaling while still being slower in terms of absolute time to solution. Absolute runtime or performance should therefore always be considered together with speedup.

(c) Case (s=0) and (c(N)=α N)

Now consider the special case

$$s = 0, \qquad p = 1, \qquad c(N) = \alpha N,$$

where (α) is a dimensionless constant.

The speedup becomes

$$S(N, \sigma) = \frac{1}{\frac{1}{N} + \sigma\alpha N}.$$

Equivalently,

$$\boxed{S(N, \sigma) = \frac{N}{1 + \sigma\alpha N^2}}.$$

Let (P(1)) denote the single-worker performance of the original, non-slowed program. After applying the slowdown factor (σ), the single-worker computational performance becomes

$$P(1, \sigma) = \sigma P(1).$$

Therefore, the performance using (N) workers is

$$P(N, \sigma) = P(1, \sigma) S(N, \sigma).$$

Substituting the expressions above gives

$$P(N, \sigma) = \sigma P(1) \frac{N}{1 + \sigma\alpha N^2}.$$

Hence,

$$\boxed{P(N,\sigma) = \frac{\sigma N P(1)}{1+\sigma\alpha N^2}}.$$

For sufficiently large (N), the communication term dominates:

$$\sigma\alpha N^2 \gg 1.$$

This corresponds to

$$N \gg \frac{1}{\sqrt{\sigma\alpha}}.$$

In this limit,

$$P(N,\sigma) \approx \frac{\sigma N P(1)}{\sigma\alpha N^2}.$$

The factor $(\sigma) cancels, giving$

$$\boxed{P(N,\sigma) \approx \frac{P(1)}{\alpha N} \qquad (N \gg 1).}$$

Thus, in the asymptotic communication-dominated regime, the absolute performance becomes independent of the slowdown factor (σ).

At the same time, the speedup behaves as

$$S(N,\sigma) \approx \frac{1}{\sigma\alpha N}.$$

Therefore,

$$\boxed{S(N,\sigma) \propto \frac{1}{\sigma}}$$

in the communication-dominated regime. Making the computational part slower therefore increases the measured speedup, even though the absolute parallel performance remains asymptotically unchanged.

This seemingly surprising result is physically reasonable. For very large (N), the runtime is dominated by the communication or synchronization overhead,

$$T_c(N) = \alpha N T(1),$$

which was assumed to remain unchanged when the computation is slowed down. Once this overhead dominates the total execution time, changing the computational speed has almost no effect on the actual parallel runtime. However, the single-worker reference runtime becomes larger by a factor $(1/\sigma)$, so the calculated speedup appears better.

The crossover between the computation-dominated and communication-dominated regimes occurs approximately when

$$\frac{1}{N} \approx \sigma\alpha N,$$

which gives

$$\boxed{N_{\text{cross}} \approx \frac{1}{\sqrt{\sigma\alpha}}}.$$

Thus, reducing (σ) shifts the apparent scaling limit to a larger number of workers. This demonstrates why slowing down a program can make its scaling curve look better without making the parallel implementation intrinsically better.

In summary,

$$\boxed{S(N,\sigma) = \frac{1}{s + \frac{p}{N} + \sigma c(N)}}$$

and, for (s=0) and (c(N)=α N),

$$\boxed{P(N,\sigma) = \frac{\sigma N P(1)}{1 + \sigma\alpha N^2} \overset{N\gg 1}{\approx} \frac{P(1)}{\alpha N}.}$$

Hence, a slower program can exhibit a larger speedup, while its absolute performance does not improve. This is why speedup alone can be a misleading metric for evaluating the quality of parallel code.

3 Wavefront Gauss-Seidel (25 Credits)

The Gauss–Seidel update considered here is a 3D 7-point stencil of the form

$$x_{i,j,k} = \frac{1}{6}\left(x_{i-1,j,k} + x_{i+1,j,k} + x_{i,j-1,k} + x_{i,j+1,k} + x_{i,j,k-1} + x_{i,j,k+1}\right).$$

The value $x_{i-1,j,k}$ has already been updated in the previous iteration of the inner loop. Hence, there is a loop-carried dependency from one lattice update to the next.

(a) Maximum in-core performance per core

The compiler is assumed to replace the division by 6.0 with a multiplication by its reciprocal. Furthermore, all operations that are independent of the loop-carried dependency are assumed to overlap perfectly.

The five stencil contributions that do not depend on $x_{i-1,j,k}$ can be computed independently. Once the result of the previous lattice update is available, the critical dependency chain consists of

$$\text{ADD} \rightarrow \text{MUL}.$$

The scalar FP64 latencies are

$$T_{\text{ADD}} = 4 \text{ cycles}, \qquad T_{\text{MUL}} = 4 \text{ cycles}.$$

Therefore, the recurrence latency is

$$T_{\text{dep}} = T_{\text{ADD}} + T_{\text{MUL}} = 4 + 4 = 8 \text{ cycles/LUP}.$$

Thus, the maximum in-core performance is

$$P_{\max} = \frac{1}{T_{\text{dep}}} = \frac{1}{8} \text{ LUP/cy}.$$

Hence,

$$\boxed{P_{\max} = 0.125 \text{ LUP/cy per core}}.$$

(b) Can an 18-core ccNUMA domain saturate memory bandwidth?

At a clock frequency of

$$f = 2.4 \text{ GHz},$$

the maximum performance of one core is

$$P_{\max,1} = 0.125 \ \frac{\text{LUP}}{\text{cy}} \cdot 2.4 \times 10^9 \ \frac{\text{cy}}{\text{s}} = 0.3 \times 10^9 \text{ LUP/s}.$$

Thus,

$$P_{\max,1} = 0.3 \text{ GLUP/s}.$$

For 18 cores, assuming ideal scaling until the memory bandwidth limit is reached,

$$P_{\max,18} = 18 \cdot 0.3 = 5.4 \text{ GLUP/s}.$$

The optimal code balance of the Gauss–Seidel stencil is

$$B_c = 16 \text{ Byte/LUP}.$$

Therefore, a performance of 5.4 GLUP/s would require a memory bandwidth of

$$b_{\text{req}} = P_{\max,18} B_c = 5.4 \times 10^9 \cdot 16 = 86.4 \times 10^9 \text{ Byte/s}.$$

Hence,

$$b_{\text{req}} = 86.4 \text{ GB/s}.$$

Since the available memory bandwidth of one 18-core ccNUMA domain is only

$$b_{\text{mem}} = 75 \text{ GB/s},$$

we have

$$86.4 \text{ GB/s} > 75 \text{ GB/s}.$$

Therefore, the aggregate in-core performance of the 18 cores is sufficient to saturate the memory bandwidth:

Yes, an 18-core ccNUMA domain can saturate the memory bandwidth.

Equivalently, the bandwidth-limited performance is

$$P_{\text{mem}} = \frac{75 \text{ GB/s}}{16 \text{ Byte/LUP}} = 4.6875 \text{ GLUP/s}.$$

Since

$$P_{\max,18} = 5.4 \text{ GLUP/s} > 4.6875 \text{ GLUP/s},$$

the execution becomes memory-bandwidth bound before all 18 cores reach their individual in-core performance limit.

The approximate number of cores required to reach bandwidth saturation is

$$N_{\text{sat}} = \frac{P_{\text{mem}}}{P_{\max,1}} = \frac{4.6875}{0.3} = 15.625.$$

Thus, approximately

$$\boxed{N_{\text{sat}} \approx 16 \text{ cores}}$$

are sufficient to saturate the memory bandwidth under the stated assumptions.